SAIL-PROXY Developer Guide - Chapter 4

st-gr

2025-01-28

SAIL-PROXY Developer Guide

Multi-provider AI Gateway for SAP AI Core - Developer Documentation **Author:** *st-gr*

<< Previous Chapter | Content Table | Next Chapter >>

Running & Testing the Gateway

Local Development

Starting the Gateway Service

From project root:

```
# Start gateway in development mode
pnpm run dev:gateway

# With specific environment
NODE_ENV=development pnpm run dev:gateway

# With debug logging
DEBUG=sail-proxy:* pnpm run dev:gateway
```

From service directory:

```
cd services/gateway
pnpm run dev

# Production build and start
pnpm run build
pnpm start
```

Stopping Stuck Processes

Kill processes on Gateway port (adapted from project docs):

```
sudo kill -9 $(sudo lsof -t -iTCP:3000 -sTCP:LISTEN)
```

Comprehensive port cleanup:

```
# Check what's using the port
lsof -i :3000

# Kill all Node.js processes (use with caution)
pkill -f node
```

```
# Or kill specific process by PID
kill -9 <process-id>
```

Testing Framework

Test Structure (from `/CLAUDE.md`)

Gateway test organization:

```
services/gateway/test/
├── setupTests.ts              # Global test setup
├── clients/                   # Client integration tests
├── config/                    # Configuration tests
├── integration/               # Integration tests (formerly scripts/)
└── usage-tracking*.test.ts    # Usage tracking tests
```

Running Tests

All gateway tests:

```
# From project root
pnpm test:gateway

# Specific test categories
pnpm test:gateway:unit         # Unit tests only
pnpm test:gateway:integration  # Integration tests only
pnpm test:gateway:usage        # Usage tracking tests only

# From gateway service directory
cd services/gateway
pnpm test                      # All gateway tests
pnpm test:unit                 # Unit tests only
pnpm test:integration          # Integration tests only
```

Watch mode for development:

```
pnpm test:watch
# Or from gateway directory
cd services/gateway && pnpm test --watch
```

Coverage reports:

```
pnpm test:coverage
```

Test Configuration

Jest configuration (`services/gateway/jest.config.js`):

```
module.exports = {
  preset: 'ts-jest/presets/default-esm',
  extensionsToTreatAsEsm: ['.ts'],
  globals: {
    'ts-jest': {
      useESM: true
    }
  },
  testEnvironment: 'node',
  roots: ['<rootDir>/src', '<rootDir>/test'],
  testMatch: ['**/__tests__/**/*.ts', '**/?(*.)+(spec|test).ts'],
  transform: {
    '^.+\\.ts$': 'ts-jest'
  },
  setupFilesAfterEnv: ['<rootDir>/test/setupTests.ts'],
  collectCoverageFrom: [
```

```
    'src/**/*.ts',
    '!src/**/*.d.ts',
    '!src/**/index.ts'
  ]
};
```

Shared Test Utilities (from `/CLAUDE.md`)

Using shared test utilities:

```
import {
  TestDataFactory,
  TestAssertions,
  TestMocks,
  setupTestEnvironment,
  teardownTestEnvironment
} from '@sap-llm-gateway/libs/test-utils';

describe('Gateway API Tests', () => {
  beforeEach(async () => {
    await setupTestEnvironment();
  });

  afterEach(async () => {
    await teardownTestEnvironment();
  });

  it('should validate API key format', () => {
    const apiKey = TestDataFactory.createApiKeyRequest();
    const isValid = TestAssertions.isValidApiKeyFormat(apiKey.token);
    expect(isValid).toBe(true);
  });

  it('should handle mock responses', async () => {
    const mockResponse = TestMocks.createMockHttpResponse({
      models: ['gpt-4o', 'claude-3-5-sonnet']
    }, 200);

    // Test with mock response
    expect(mockResponse.status).toBe(200);
  });
});
```

API Testing

Manual API Testing

Test model listing:

```
# Test health endpoint
curl -X GET http://localhost:3000/health

# Test model listing (requires valid API key)
curl -X GET http://localhost:3000/v1/models \
  -H "Authorization: Bearer sp-proj-your-api-key"

# Test OpenAI endpoint
curl -X POST http://localhost:3000/openai/v1/chat/completions \
  -H "Authorization: Bearer sp-proj-your-api-key" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-4o",
    "messages": [{"role": "user", "content": "Hello"}],
```

```
    "max_tokens": 50
  }'
```

Test streaming:

```
curl -X POST http://localhost:3000/openai/v1/chat/completions \
  -H "Authorization: Bearer sp-proj-your-api-key" \
  -H "Content-Type: application/json" \
  -N \
  -d '{
    "model": "gpt-4o",
    "messages": [{"role": "user", "content": "Count to 10"}],
    "stream": true
  }'
```

Test Anthropic endpoint:

```
curl -X POST http://localhost:3000/anthropic/v1/messages \
  -H "x-api-key: sp-proj-your-api-key" \
  -H "Content-Type: application/json" \
  -H "anthropic-version: 2023-06-01" \
  -d '{
    "model": "claude-3-5-sonnet-20241022",
    "messages": [{"role": "user", "content": "Hello"}],
    "max_tokens": 100
  }'
```

Integration Testing

Example integration test:

```
// services/gateway/test/integration/openai-api.test.ts
import request from 'supertest';
import { app } from '../../src/app';
import { TestDataFactory } from '@sap-llm-gateway/libs/test-utils';

describe('OpenAI API Integration', () => {
  let apiKey: string;

  beforeAll(async () => {
    const keyData = TestDataFactory.createApiKeyRequest();
    apiKey = keyData.token;
  });

  describe('POST /openai/v1/chat/completions', () => {
    it('should handle valid chat completion request', async () => {
      const response = await request(app)
        .post('/openai/v1/chat/completions')
        .set('Authorization', `Bearer ${apiKey}`)
        .send({
          model: 'gpt-4o',
          messages: [{ role: 'user', content: 'Hello' }],
          max_tokens: 10
        })
        .expect(200);

      expect(response.body).toHaveProperty('id');
      expect(response.body).toHaveProperty('choices');
      expect(response.body.choices[0]).toHaveProperty('message');
    });

    it('should handle streaming requests', async () => {
      const response = await request(app)
        .post('/openai/v1/chat/completions')
        .set('Authorization', `Bearer ${apiKey}`)
        .send({
```

```
          model: 'gpt-4o',
          messages: [{ role: 'user', content: 'Hello' }],
          stream: true
        })
        .expect(200);

      expect(response.headers['content-type']).toBe('text/event-stream');
    });

    it('should reject invalid API key', async () => {
      await request(app)
        .post('/openai/v1/chat/completions')
        .set('Authorization', 'Bearer invalid-key')
        .send({
          model: 'gpt-4o',
          messages: [{ role: 'user', content: 'Hello' }]
        })
        .expect(401);
    });
  });
});
```

Performance Testing

Load Testing

Using artillery.js:

```
# artillery-config.yml
config:
  target: 'http://localhost:3000'
  phases:
    - duration: 60
      arrivalRate: 10
  defaults:
    headers:
      Authorization: 'Bearer sp-proj-your-test-key'
      Content-Type: 'application/json'

scenarios:
  - name: 'Chat Completions'
    weight: 100
    flow:
      - post:
          url: '/openai/v1/chat/completions'
          json:
            model: 'gpt-4o'
            messages:
              - role: 'user'
                content: 'Hello world'
            max_tokens: 50
```

Run load test:

```
npm install -g artillery
artillery run artillery-config.yml
```

Memory and CPU Profiling

Node.js profiling:

```
# CPU profiling
node --prof services/gateway/dist/index.js
```

```
# Heap profiling
node --inspect services/gateway/dist/index.js

# Memory usage monitoring
node --trace-gc services/gateway/dist/index.js
```

Memory leak detection:

```
// Add to test setup
process.on('warning', (warning) => {
  console.warn('Node.js Warning:', warning);
});

// Monitor memory usage in tests
const getMemoryUsage = () => {
  const usage = process.memoryUsage();
  return {
    rss: Math.round(usage.rss / 1024 / 1024),
    heapTotal: Math.round(usage.heapTotal / 1024 / 1024),
    heapUsed: Math.round(usage.heapUsed / 1024 / 1024),
    external: Math.round(usage.external / 1024 / 1024)
  };
};
```

Debugging

Debug Configuration

VS Code debugging (`.vscode/launch.json`):

```
{
  "configurations": [
    {
      "name": "Debug Gateway",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/services/gateway/src/index.ts",
      "env": {
        "NODE_ENV": "development",
        "DEBUG": "sail-proxy:*"
      },
      "runtimeArgs": ["--loader", "tsx/esm"],
      "skipFiles": ["<node_internals>/**"],
      "console": "integratedTerminal"
    },
    {
      "name": "Debug Gateway Tests",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/node_modules/.bin/jest",
      "args": ["--runInBand", "--no-cache"],
      "cwd": "${workspaceFolder}/services/gateway",
      "console": "integratedTerminal"
    }
  ]
}
```

Logging Configuration

Debug logging:

```
import debug from 'debug';

const log = debug('sail-proxy:gateway');
```

```
const logAuth = debug('sail-proxy:auth');
const logAPI = debug('sail-proxy:api');

// Usage in code
log('Gateway started on port %d', port);
logAuth('Authenticating request %s', req.correlationId);
logAPI('SAP AI Core request: %O', sapRequest);
```

Structured logging:

```
import { logger } from '../utils/logger';

logger.info('Request processed', {
  correlationId: req.correlationId,
  userId: req.user.id,
  endpoint: req.path,
  duration: Date.now() - req.startTime,
  tokensUsed: response.usage.total_tokens
});
```

Continuous Integration

GitHub Actions (if configured)

Calls from GitHub-hosted runners to SAP AI Core can stall (the connection opens, then no response arrives), while the same calls from inside a Kyma cluster do not. The CI pipeline can therefore route the gateway's SAP traffic through a small relay that you run in your own Kyma cluster: set the repository secret `SAP_RELAY_BASE`, and the gateway started by the pipeline uses the relay for every SAP call, including the deployment URLs SAP returns (`SAP_DEPLOYMENT_URL_BASE`). Without the secret, nothing changes and SAP is reached directly. Setup, operation and key rotation are in `ci/sap-relay/README.md`; the pipeline side is described in `ci/CI_PIPELINE.md`.

Test workflow (`.github/workflows/test-gateway.yml`):

```
name: Gateway Tests

on:
  push:
    paths: ['services/gateway/**']
  pull_request:
    paths: ['services/gateway/**']

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: '20'
          cache: 'pnpm'

      - name: Install dependencies
        run: pnpm install

      - name: Run Gateway tests
        run: pnpm test:gateway
        env:
          NODE_ENV: test

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          file: ./services/gateway/coverage/lcov.info
```

Pre-commit Hooks

Test automation (`.husky/pre-commit`):

```
#!/bin/sh
. "$(dirname "$0")/_/husky.sh"

# Run tests for changed files only
changed_files=$(git diff --cached --name-only)

if echo "$changed_files" | grep -q "services/gateway/"; then
  echo "Running Gateway tests..."
  pnpm test:gateway:unit
fi
```

Environment-Specific Testing

Test Environments

Development testing:

```
NODE_ENV=development pnpm test:gateway
```

CI testing:

```
NODE_ENV=test CI=true pnpm test:gateway --coverage --watchAll=false
```

Integration testing with real SAP AI Core:

```
# Use separate test environment
NODE_ENV=integration \
SAP_TEST_CLIENT_ID=your-test-client \
SAP_TEST_CLIENT_SECRET=your-test-secret \
pnpm test:gateway:integration
```

Next: Explore the Admin Cockpit (CAP) service implementation and testing.